A határ alkotmánya: szintek, életképesség és metanyelv

Szőke László

2026. október 30. · frissítve: 2026. november 6. · közreműködés: Claude (Anthropic) · a hivatkozások ellenőrizve: 2026. november 1. (az új nevek forrásai: 2026. november 6.)

Összefoglaló

A rendszer biztonságos kimenete kikényszeríthető kívülről, a belső szabályok ismerete nélkül, egy kijelölt határon és kimondott előfeltevések mellett; több szinten ehhez minden szintnek saját alkotmány kell, és a bizonyítás a metanyelven történik.

- **Egy szinten** a kapu-tétel két esetből bizonyítható: a kimenet mindig a megengedett halmazban marad, bármi van a dobozban.
- **A sejtek helyi szabályaiból nem következik a közös szabály** (ellenpélda: $4 + 5 > 8$). Kvóta-szerződéssel következik (bizonyított), de csak ha minden szinten minden kapu áll.
- **A közösségi életképesség** lehetetlen lehet (3-3-3 minimum, 8 készlet), ilyenkor az alkotmánynak előnyt kell kimondania: értékválasztás, nem matematika.
- **Az alkalmazkodás sebessége** nem lehet egyetlen szereplőnél összpontosítva (koncentrációs korlát, bizonyítva).
- **A metanyelv** a szabályokról állít, nem parancs: azért használható a `.bio`-ra, mert az eldönthető, és a szintek rétegezése elkerüli az önhivatkozást.
- Egy eszközön (MetaSpace Studio) a lánc végigvihető, mérve: leírás, gépi ellenpélda, kapu három nyelven, igazolási dosszié. A gép a hiányzó szabályt csak megnevezett veszélynél találja meg, és a kapu csak a kijelölt határon kötelez.

Gyakorlati következmény (vibe-coding). Ha a programot mesterséges intelligencia írja, a programnyelv ismerete nélkül is garantálható, hogy a világ csak megengedett hatást kap, feltéve hogy a hatást kapu kényszeríti ki, a rendszeren kívül. Nem spórolható meg a szakterület ismerete (mi a megengedett), és a kapu nem biztosítja, hogy a doboz valamit meg is tesz.

Amit nem állítunk: hogy a `.bio` a világ törvénykönyve, hogy a modell a valóság, vagy hogy a cél helyessége bizonyítható. A „100%” itt azt jelenti, hogy minden előfeltevés ki van mondva, és a következtetés ezekből szükségszerű.

Hatókör és szerénység (2026-11-06). (1) A kapu-tétel egyszerű lemma: egy ellenőrzés és egy tartalék érték. Nem új: a szakirodalom futásidejű kikényszerítésnek (Schneider, 2000), biztonsági pajzsnak (Alshiekh és mások, 2018), Simplex-architektúrának (Sha, 2001) és referencia-figyelőnek (Anderson, 1972) hívja. (2) A kapu a **biztonságot** (a kimenet nem lép ki E-ből) garantálja, a **helyességet** (a program el is végzi a dolgát) nem; a futásidejű őrök általában csak biztonsági tulajdonságot tudnak kikényszeríteni (Schneider, 2000; Lamport, 1977). (3) Időben összefüggő szabálynál (például az esés sebessége) a tétel egyetlen értékre nem elég: a biztonságos értékre cserélés átmenete is számít (a 40-ről 60-ra ugrás sérti a 2/mp szabályt; lásd az életképesség-táblát). (4) A nyelv–program hasonlat **szerkezeti analógia**, nem izomorfia és nem bizonyíték: a nyelvtan leírás, a kód ok. (5) Az eszköz (MetaSpace Studio) szerzője a tanulmány szerzője; független ellenőrzése nincs.

Közreműködés (2026-11-06). A tanulmány és a hozzá tartozó előadás-anyag Claude-dal, az Anthropic nyelvi modelljével közösen készült. A tézist, a filozófiai keretet, a MetaSpace-eszközt, a döntéseket és a felelősséget a szerző adta; a szöveg és a kód nagy részét a modell fogalmazta és írta. A főelőadást és a kiegészítőt a szerző dianként hagyta jóvá; a 2026. november 6-i átdolgozás (szószedet, jegyzetek, ez a bővítés, javítások) a szerző irányával, de diankénti jóváhagyás nélkül készült. A szakma szerint (COPE, ICMJE) az MI nem lehet szerző, a használatát közzé kell tenni, a felelősség az emberé. A szerző a megrendelő és felelős szerkesztő, a modell a generátor; ez nem tiszta vibe-coding, mert a szerző specifikált, irányított és jóváhagyott. A munka maga is próbája a tételnek: a megengedett halmazt (P1) az ember adta, de független kapu (P4) nem volt, a modell a saját korábbi kimenetét auditálta, és az audit hét túlállítást talált, amelyek addig átmentek a modellen és a jóváhagyáson; a független kapu egy külső emberi bírálat volt. Hogy a szerző a generált kódból mennyit olvasott át soronként, azt a szerző mondja ki. Részletek: kozremukodes.html.

Szószedet

Minden fogalom egyetlen jelentésben szerepel a dokumentumban. A szószedet azt is megmondja, honnan jön a szó (filozófia, nyelv, biológia), mi a bevett informatikai neve, és hol nem szabad szó szerint érteni. A hasonlatok szerkezeti analógiák: szemléltetnek, nem bizonyítanak. Ugyanez a szószedet külön oldalon is elérhető (szotar.html).

Kifejezés	Itt jelentése	Bevett informatikai név (és eredet)	Nem azonos azzal / hol sántít
Rendszer	A világ egy elkülönített része, amelynek van határa.	komponens; szolgáltatás; program a környezetével. Eredet: rendszerelmélet.	nem a világ egésze (annak nincs külseje, ezért nincs hová kaput állítani)
Határ	Azon változók összessége, amelyeken át a program és a környezete egymásra hat (pacemakerben: a kimenő pulzus).	interfész; bizalmi határ (trust boundary). Eredet: rendszerelmélet (Ashby, Bernard); biológia (sejthártya).	nem a program fizikai burka; nem a teljes világ (annak nincs külseje)
Doboz	A program belseje, amelyről semmit sem tételezünk fel.	megvalósítás (implementáció); fekete doboz (black box). Eredet: rendszerelmélet (fekete doboz).	nem azt állítjuk, hogy ismerhetetlen, hanem azt, hogy a bizonyítás nem használja fel
Megengedett halmaz (E)	A határ azon értékeinek halmaza, amelyeket a szabály megenged (pacemakerben: 40–165 ütés/perc).	biztonsági burkoló (safe set, safety envelope); engedélyezési lista (allowlist). Eredet: matematika (halmaz).	nem az optimális sáv; nem az összes lehetséges érték
Optimális sáv	E kitüntetett része: a cél tartománya (-tól -ig).	üzemi tartomány (operating range) a biztonsági burkolón belül. Eredet: szabályozáselmélet.	nem egyetlen pont; az E-nél szűkebb, és más kérdés: a hasznosság, nem a biztonság
Kapu	A határon álló, a programtól független őr: a megengedett értéket átengedi, a többit előre kijelölt biztonságos értékre cseréli.	kimeneti őr: futásidejű figyelő (runtime monitor), biztonsági pajzs (safety shield), kimenő szűrő (egress filter). Eredet: saját szó.	nem tűzfal (az főleg a bejövő forgalmat szűri); nem teszt; nem a program belsejéről dönt, csak a kimenetről
Biztonságos érték (s)	Az E-nek előre kijelölt eleme, amelyre a kapu cserél (pacemakerben: 60 ütés/perc).	biztonságos állapot (safe state); tartalék érték (fallback); hibabiztos állapot (fail-safe). Eredet: saját szó.	időben összefüggő szabálynál az s-hez vezető átmenet is számít: 150-ről 60-ra ugrani sérti a „legfeljebb 2 ütés/perc/mp” szabályt
Hatás és ok	A hatás: ami a határon kilép és a világot éri. Az ok: a parancs vagy a belső döntés, amely ezt kiváltotta. A kapu a hatást ítéli meg.	kimenet-alapú ellenőrzés (utófeltétel, postcondition) szemben a bemenet- vagy kódalapú ellenőrzéssel. Eredet: okságfilozófia.	a „hatás” itt a határon mérhető kimenet, nem a világ minden következménye
Zárt világ elve	A kapu azt sorolja fel, ami megengedett. Ami a szabályban nincs, az a határon át nem jut ki.	alapértelmezett tiltás (default-deny); engedélyezési lista (allowlist); hibabiztos alapértelmezés (fail-safe defaults). Eredet: logika (Reiter, 1978).	nem tiltólista (az kifelejti, amire nem gondoltunk); nem a fizikai világ tulajdonsága
Mellékút	Minden út, amelyen a hatás a kapu megkerülésével jut ki.	megkerülés (bypass); a teljes közvetítés hiánya (complete mediation, Saltzer–Schroeder, 1975). Eredet: saját szó.	a kapu-állítás csak akkor érvényes, ha nincs mellékút: ezt a rendszer szintjén kell kikényszeríteni
Megkerülhetetlenség (rendszermag-szint, DCC Ring 0)	A kapu kikényszerítése a rendszermagban, hogy a program ne tudja kijátszani.	referencia-figyelő (reference monitor, Anderson, 1972): mindig meghívott, kijátszhatatlan, ellenőrizhető. Eredet: MetaSpace (DCC).	mért állapot (a védelmi ledger szerint): fájlírásra, egy eldobható virtuális gépen (Linux 6.1, 2026-08-07-i építés); nincs üzembe helyezve; root ellen nem véd; független ellenőrzés nincs
Tudni-hogyan és tudni-hogy	Tudni-hogyan: helyesen cselekedni a szabály kimondása nélkül. Tudni-hogy: a szabályt kimondani.	implicit tudás szemben a specifikációval. Eredet: filozófia (Ryle, 1949).	a kettő nem zárja ki egymást; a nyelvtan nélkül is helyesen beszélünk, de ettől a nyelvtan nem felesleges
Hallgatólagos tudás	Amit tudunk, de nem tudunk szabályként kimondani (a „Látom a kutyát” helyességét felismerjük).	felismerő (osztályozó, oracle) szemben az előállítóval (generátorral). Eredet: filozófia (Polanyi, 1966).	a kapunak felismerni kell a helyeset, nem előállítani; ezért elég egy halmaz-ellenőrzés, nem kell hozzá a program ismerete

Kifejezés	Itt jelentése	Bevett informatikai név (és eredet)	Nem azonos azzal / hol sántít
Szerkezeti analógia	Két terület hasonló szerkezetű: a megfeleltetés a viszonyokra áll, nem a részletekre. Szemléltet, nem bizonyít.	Eredet: filozófia (analógia, szerkezetleképezés).	nem izomorfia (az kölcsönösen egyértelmű, szerkezetmegőrző leképezés); legfeljebb részleges homomorfia. A nyelvtan leírás, a kód ok: ez az eltérés
Többszörös megvalósíthatóság	Ugyanaz a helyes viselkedés többféle belsővel is létrejöhet.	interfész és megvalósítás szétválasztása. Eredet: elmefilozófia (Putnam, Fodor).	a helyes viselkedés a kimeneten ítélhető meg; a belső ettől független (de a belső okozza a viselkedést)
A beszélő-közösség mint kapu	A gyerek a kimenetén át tanul: a közösség kijavítja. A belsejét senki nem vizsgálja.	külső visszacsatolás; kimeneti ellenőrzés (output validation). Eredet: nyelvelsajátítás.	a közösség sem tévedhetetlen; a nyelvnél nincs kimondott E, a pacemakernél van
Alkotmány	A határra vonatkozó, véges, kimondott, megszeghetetlen szabályok: a megengedett halmaz és a vészállapot.	invariáns (mindig fennálló feltétel); biztonsági tulajdonság; specifikáció. Eredet: jog- és politikafilozófia (hasonlat).	metafora: a pontos fogalom az invariáns. Az alkotmány azt mondja meg, mi nem történhet meg; a törvény (leírás) azt, mi történik
Törvény (leíró)	A működés leírása: mi történik (a glükóz lebomlik).	modell; viselkedésleírás. Eredet: természettudomány.	nem előírás; ez az alkotmány szerepe
Tökéletes	A kimenet minden elérhető helyzetben E-ben van. A célhoz viszonyított teljesség.	biztonsági tulajdonság teljesül (safety). Eredet: arisztotelészi filozófia (teleios).	nem hibátlan program; nem optimális; nem azt jelenti, hogy a program el is végzi a dolgát
Százszázalékos	Minden előfeltevés ki van mondva, és a következtetés ezekből szükségszerű.	feltételes garancia (assume–guarantee). Eredet: logika.	nem azt jelenti, hogy a valóságban semmi nem romolhat el
Biztonság (safety)	„Valami rossz soha nem történik meg”: például a kimenet soha nincs E-n kívül.	biztonsági tulajdonság (safety property). Eredet: számítástudomány (Lamport, 1977; Alpern–Schneider, 1985).	ezt a kapu kikényszeríti. A futásidejű őrök általában csak ilyen tulajdonságot tudnak kikényszeríteni (Schneider, 2000)
Élősség (liveness)	„Valami jó előbb-utóbb megtörténik”: a pacemaker ténylegesen ütemet ad.	élősségi tulajdonság (liveness property). Eredet: számítástudomány (Lamport, 1977; Alpern–Schneider, 1985).	ezt a kapu nem garantálja: egy mindig 60-at adó program is átmegy rajta
Helyesség	A program azt teszi, amire való.	funkcionális helyesség (functional correctness). Eredet: informatika.	nem azonos a biztonsággal; ehhez a szakterület ismerete és tesztelés kell
Lemma („kaputétel”)	Ha y eleme E-nek, a kapu átengedi, különben s-t ad, s pedig eleme E-nek: a kimenet mindig E-ben marad.	a futásidejű kikényszerítés alapállítása; egy ellenőrzés és egy tartalék érték. Eredet: matematika.	egyszerű, nem ez az újdonság. Időben összefüggő szabálynál indukció és életképesség kell hozzá
Bizonyított	A modellre és a kimondott előfeltevésekre igaz.	formálisan ellenőrzött (adott modellre). Eredet: logika.	három különböző dolgot is jelent, ne keverd: (1) matematikai lemma, (2) Z3 PASS, vagyis a szabályok ellenőrzése, (3) mért futás. Egyik sem azt jelenti, hogy a valóságban igaz
PASS / FAIL (Z3)	A szabályok ellentmondásmentesek-e, és kizárják-e a megnevezett veszélyt. FAIL esetén egy konkrét ellenpéldát kapunk.	SMT-megoldó eredménye (sat / unsat). Eredet: formális módszerek.	a szabályokat vizsgálja, a programot nem; a hiányzó, meg nem nevezett veszélyt nem találja meg
Ellenpélda	Egy konkrét eset, amely az állítást megcáfolja (például 40 ütés/perc és 3 ütés/perc/mp esés).	ellenpélda (counterexample); az SMT-ben egy kielégítő modell. Eredet: logika.	nem statisztikai kivétel
Veszély (HAZARD)	Egy megnevezett, elkerülendő helyzet. A gép azt vizsgálja, hogy a szabályok kizárják-e.	veszélyforrás (hazard). Eredet: biztonságtechnika.	a veszélyt az ember nevezi meg; amit senki nem nevezett meg, azt a gép nem ellenőrzi
Specifikáció (.bio)	A határ leírása: jelek, szabályok, biztonságos állapot.	formális specifikáció; szerződés (contract). Eredet: informatika.	a specifikáció írása is programozás, de nem a megvalósításé; ezért mondjuk, hogy a megengedett halmazhoz a szakterület kell, a programnyelv nem

Kifejezés	Itt jelentése	Bevett informatikai név (és eredet)	Nem azonos azzal / hol sántít
Igazolási dosszié	A Studio által készített napló: a specifikáció lenyomata, az eredmények, a hatókör.	bizonyíték-csomag (assurance case) – de önmagunk készítette. Eredet: MetaSpace Studio.	nem tanúsítvány, nem független ellenőrzés
Bizalmi mag	Az, amiben meg kell bíznunk: az ellenőrző és a kódgenerátor.	megbízható számítási alap (TCB). Eredet: biztonságtechnika.	ha az ellenőrző hibás, a bizonyítás sem megbízható (P4)
Eldönthető	Véges lépésben mindig megadható az igen/nem válasz.	eldönthető (decidable). Eredet: számításelmélet.	nem azt jelenti, hogy gyors
Életképesség (kiút-feltétel)	Bármely E-beli helyzetből van olyan következő lépés, amely E-ben tart, és onnan is van ilyen.	életképességi mag (viability kernel); visszaállíthatóság. Eredet: rendszerelmélet (Aubin).	nem az, hogy az állapot éppen E-ben van. Pacemakerben a 40→60 ugrás egy lépésben sérti a 2/mp szabályt
Szint	A rendszerek rétege: az alsó szint rendszerei a felsőnek részei.	rétegzés; összetétel (composition). Eredet: rendszerelmélet.	nem rangsor. Nem azonos a Studio „fok” szavával
Fok (1., 2., 3.)	A kikényszerítés erőssége a Studióban.	kikényszerítési erősség (enforcement tier). Eredet: MetaSpace Studio.	nem a rendszerek rétege (ez a „szint”)
Garancia–feltevés	Az alsó szint garantál valamit, feltéve hogy a környezete teljesít valamit.	garancia–feltevés szerződés (assume–guarantee contract). Eredet: formális módszerek.	nem bizalom vagy hit
Tárgynyelv és metanyelv	A tárgynyelven a szabályokat mondjuk ki (.bio); a metanyelven a szabályokról állítunk (például hogy ellentmondásmentesek).	specifikációs nyelv és ellenőrző (verifier). Eredet: logika (Tarski).	nem a tárgynyelv „fölötti parancs”
Vibe-coding	A programot mesterséges intelligencia írja, az ember a kívánt viselkedést mondja el, a kódot nem olvassa végig.	MI-generált kód átnézés nélkül. Eredet: szleng (Karpathy, 2025).	nem azt jelenti, hogy a hatás ellenőrzése felesleges – éppen azt kell kikényszeríteni
Szerző és MI viszonya	A szerző a megrendelő és specifikáló, a felelős szerkesztő; az MI a generátor és végrehajtó. Az ember adja a megengedett halmazt (P1), és viseli a felelősséget.	megrendelő és kivitelező; megbízó és ügynök (principal és agent); ember a körben (human-in-the-loop). Eredet: kiadói etika (COPE, ICMJE); szoftvermérnökség.	nem szerzőtárs: az MI nem lehet szerző, de a használatát közzé kell tenni. A tyúk–tojás kérdés akkor válik valódi problémává, ha az MI ellenőrzi az MI-t (nincs független kapu)
Vibe-coder	Aki MI-vel íratja a programot, és a kódot nem olvassa végig.	specifikáló és jóváhagyó (az MI-asszisztált fejlesztésben). Eredet: szleng (Karpathy, 2025).	ebben a munkában a szerző nem tiszta vibe-coder: specifikált, irányított, a főelőadást és a kiegészítőt dianként jóváhagyta (a 2026-11-06-i átdolgozás kivételével)
Független ellenőrzés	Az ellenőrző olyan féltől jön, aki sem a doboz (a kód) írója, sem a szabályt kimondó. Ebben a munkában nincs; csak egy külső emberi bírálat vizsgálta.	független ellenőrzés és validálás (IV&V). Eredet: biztonságtechnika (a bizonyítás függetlensége).	az MI-vel végzett önellenőrzés nem független: ugyanaz a modellcsalád hasonló hibákat követhet el

Az utolsó sor különösen fontos: a „bizonyított” a modellre vonatkozik. Hogy a modell a valóság, az megfigyelés kérdése.

Leírás és megszabás: a nyelvtan és a sejthártya

A törvény azt mondja meg, mi történik; az alkotmány azt, mi nem történhet meg. A `.bio` az utóbbi nyelve.

Az, aki az anyanyelvét nyelvtan nélkül beszéli, a viselkedést tudja, a szabályt nem. Ryle ezt „tudni, hogyan” és „tudni, hogy” különbségének nevezte, Polanyi hallgatólagos tudásnak. A nyelvtan utólagos leírás: a nyelv megelőzte.

A hasonlat pontos állítása. Ebből nem az következik, hogy nincs szükség nyelvtanra, hanem az, hogy a helyes viselkedéshez van másik út is: a kimenetet ítéli meg valaki, nem a belsőt (többszörös megvalósíthatóság). A gyerek nem a nyelvtanból tanul: a beszélőközösség kijavítja, vagyis a közösség egy külső kapu a kimeneten. A kapunak a helyeset **fel kell ismernie**, nem előállítania: a beszélő bármely mondatról

megmondja, hogy jól hangzik-e, a szabályt viszont nem tudja kimondani. A pacemakernél a felismerés egy intervallum. A megfeleltetés tehát a **közösség** ↔ **kapu** viszonyra erős; a **nyelvtan** ↔ **forráskód** viszonyra gyenge, mert a nyelvtan utólagos leírás, a kód pedig a viselkedés oka. Ezért szerkezeti analógiáról beszélünk, nem izomorfiáról.

A biológia ugyanezt a szerkezetet mutatja. Bernard a belső környezet állandóságát, Ashby (1952) az élőlény *lényegi változóit* írta le, amelyek csak meghatározott határok között maradhatnak. Maturana és Varela szerint az élő az a rendszer, amely a saját szerveződését tartja fenn. A sejt életének kezdete egy határ: a sejthártya, amely megmondja, mi számít belülnek és mi kívülnek. Az élet tehát maga alkotmány, és a célja (a fennmaradás) nem választott, hanem adott.

Az informatikában egy interfész-szerződés (típus, jogosultság, tűzfalszabály) ugyanúgy nem a program belsejét írja le, hanem azt szabja meg, mi léphet át a határon.

Terület	Ami működik	Megszabás (határ)	Róla szóló nyelv (metanyelv)
Nyelv	a beszéd	a nyelvi norma: mi helyes mondat	a nyelvtan
Biológia	a sejt anyagcseréje	a lényegi változók határai, a sejthártya	az élettan
IT	a program futása	az interfész-szerződés	a specifikáció és a bizonyító

Következmény: a viselkedés a szabály ismerete nélkül is lehet helyes, és ezért kikényszeríthető kívülről a biztonságos kimenet.

A tétel egy szinten

Tétel. Bármely ismeretlen belsejű dobozra, a határon álló kapuval együtt, a kimenet minden elérhető helyzetben a megengedett halmazban (E) marad.

Bizonyítás. Tetszőleges bemenetnél a doboz valamilyen y kimenetet ad. Két eset van, harmadik nincs. Ha y eleme E-nek, a kapu átengedi, tehát a kimenet E-ben van. Ha y nem eleme E-nek, a kapu a biztonságos s értéket adja, és s eleme E-nek. A bizonyítás a dobozról semmit nem használ fel: a „minden dobozra” azt jelenti, hogy a tétel akkor is igaz, ha a doboz belsejéről semmit sem tudunk.

Három pontosítás teszi hiánytalanná:

- Ha a doboz nem válaszol, az is kimenet: az időkorlát lejártakor a kapu s-t ad.
- Ha az elvárás a lépések sorrendjére is vonatkozik (például a változás sebességére), a bizonyítás teljes indukcióval megy, és kell hozzá az **életképesség**: bármely E-beli helyzetből van olyan következő lépés, amely E-n belül tart, és onnan is van ilyen, és így tovább (nincs zsákutca).
- A bizonyításokat ellenőrző program (Z3) csak az elvárást vizsgálja (ellentmondásmentes-e, van-e zsákutca), a dobozról nem állít semmit.

Példa: pacemaker (`pacemaker.bio`: pulzus 40 és 165 között, változás legfeljebb 2 / mp, vészállapot 60).

Életképesség lépcsője	Eredmény
Nincs zsákutca: bármely állapotból van tovább	bizonyított (triviálisan: az érték tartása mindig lehetséges)
A vészállapot (60) egy lépésben elérhető minden állapotból	**hamis:** 40-ről 60-ra ugrás 20 egység, a határ 2
A vészállapotig legfeljebb ennyi másodperc kell	bizonyított: 10 mp, ha csak a gyorsulás korlátozott; 53 mp, ha a lassulás is

A gyakorlati tanulság: a specifikáció a saját vészállapotával is ellentmondhat, és hiányzik belőle a határidő. Ezt a gép kimutatja.

Gépi megvalósítás. A „ha túl nagy, a felső határ; ha túl kicsi, az alsó” alakú kapu átengedi a „nem szám” értéket (a gép így jelzi az értelmezhetetlen eredményt), mert vele minden összehasonlítás hamis. A javított kapu (csak a pozitívan igazolt értéket engedi át, különben s-t ad) mind a 2^{64} bitmintára bizonyított.

Előfeltevések, amelyeken a tétel áll:

- **P1:** a cél (E) helyes. Értékválasztás, nem bizonyítható. A biológiában adott (fennmaradás).
- **P2a:** nincs mellékút, minden hatás átmegy a kapun.
- **P2b:** a határ teljes: a lényeges változók láthatók rajta, és a határon át be lehet avatkozni.
- **P3:** a fizika és a környezet együttműködik, köztük a környezet változási sebessége.
- **P4:** az ellenőrző és a fordító program helyes (bizalmi mag). Ha nem, a bizonyítás sem megbízható.

A gyakorlatban: vibe-coding kapuval

Fő tétel. A vibe-codinghoz (a programot mesterséges intelligencia írja) nem kell ismerni a programnyelvet, ha nem az okot, hanem a hatást kényszerítjük ki, a rendszeren kívül, kapuval. Ez a kapu-tétel közvetlen következménye: a tétel a doboz belsejéről semmit nem tételez fel, tehát arról sem, hogy ki és hogyan írta.

1. Te mondod ki, mi a megengedett (E).
2. A gép írja a kódot (a doboz). Nem olvassuk el, elszigetelten fut.
3. A kapu az E-ből készül, ellenőrzött generátorral, és a dobozon kívül áll.
4. A világ csak megengedett hatást kap.

Hasonlat: a biztosíték. A készülék a doboz, a felvett áram a javaslat, a biztosíték a kapu, a legfeljebb 16 amper az E, a megszakadt áramkör (0 amper) a biztonságos érték. Két eltérés: a biztosíték nem pillanatnyi (a tűrést E-be kell építeni), és ha valaki dróttal áthidalja, az mellékút (P2a).

A zárt világ elve. A kapu azt sorolja fel, ami megengedett, nem azt, ami tilos. Ami a szabályban nincs, az a határon nem létezik: nincs csatorna, amelyen hatást fejthetne ki. A tiltólistás kapu átengedte a „nem szám” értéket, amire nem gondoltunk; az engedélyezési listás (csak az megy át, ami igazolt) mind a 2^{64} esetre bizonyított. A „nem létezik” csak a határra igaz (P2a, P2b), a fizikai világra nem. Aszimmetria: ha az engedélyezési listából kimarad egy csatorna vagy érték, az azt okozza, hogy valami nem működik (látható, pótolható), nem azt, hogy kár keletkezik. Fordítva ez nem igaz: ha egy engedélyezett jelre hiányzik a korlát (például az esés sebessége), kár lehet; ezt a gép csak megnevezett veszélynél találja meg (lásd az eszközös szakaszt).

Mit ad a kapu, és mit nem.

- Amit a specifikáció a kimenetek halmazaként kimond, az a hatásban is teljesül. A hasznosságnak az a része bizonyított, amely a határon tartományként kifejezhető (az optimális sáv is ilyen, ha a környezet nem túl gyors); a többi nem.
- Nem kényszeríthető ki a pozitív kötelesség: hogy a doboz valamit meg is tegyen. Ha nem tesz semmit, az E-ben van. Ez hiány, nem veszély, és kimeneti teszttel kiderül.
- A szabály helyessége (P1) módszerfüggetlen: kézi kódolásnál ugyanúgy kell. A vibe-coding annyit változtat, hogy a kód helyessége kiesik a bizalmi magból.
- Az, hogy mi a megengedett, nem spórolható meg. Ez a szakterület ismerete és a határ pontos, formális nyelven történő kimondása; nem a megvalósítás programozása.

A soronkénti olvasás nem garancia. Ha valaki soronként írja vagy olvassa a kódot, a bizalmi mag a teljes kód. Vibe-codingnál a bizalmi mag a kapu és az ellenőrző, amely kicsi és véges. Ez a bizalmi mag méretéről szól, nem a hibák valószínűségéről.

Egy eszközön végigvíve: a MetaSpace Studio

A lánc egy eszközön végigvihető (leírás, gépi ellenpélda, kapu három nyelven, igazolási dosszié), de a gép csak azt találja meg, amit megneveztünk, és a kapu csak a kijelölt határon kötelez. A méréseket 2026-10-30-án az élő eszközön végeztük (MetaSpace Studio 2.8), a pacemaker példán. Az eszköz saját tesztjei (144) mind zöldek.

1. A hiányzó szabály csak megnevezett veszélynél derül ki. Ugyanaz a pacemaker, három futtatás:

Futtatás	Bemenet	Eredmény
1	az eredeti `pacemaker.bio`, veszély megnevezése nélkül	megfelel; a hiányzó lefelé korlátot nem látja
2	az eredeti + veszély: a kimenő ütemszám gyorsabban esik 2 ütés/perc/s-nál	hibát jelez: rés a specifikációban; ellenpélda: 40 ütés/perc, esés 3 ütés/perc/s, minden szabály teljesül, mégis veszély
3	a javított fájl + ugyanaz a veszély	megfelel; a veszélyt a lassulás-korlát bizonyítottan kizárja

Tanulság: a gép a lyukat akkor találja meg, ha a veszélyt megnevezzük. A veszélyek felsorolása szakterületi tudás, vagyis a P1 gyakorlati alakja.

2. Egy specifikáció, három nyelv, nem teljesen azonos viselkedés. A javított specifikációból Python, ipari vezérlő és C kapu generálódik, mindháromban ugyanaz az öt szabály. A viselkedés eltér: a Python kernel biztonságos értékre vált (60 ütés/perc), az ipari vezérlő és a C változat csak hibát jelez és reteszel, mert a pacemaker-fájl nem nevez meg kimeneti jelet. A határidőt és az elsőbbséget a Python csak értékként tárolja, nem kényszeríti ki. A C-kimenet ezen a gépen nincs lefordítva.

3. A nem-szám és a kerekítés. A szabályok pozitív alakúak („legalább 40 és legfeljebb 165”), a nem-szám pedig minden összehasonlításban hamis, ezért a generált kernel blokkolja (mérve: a nem-szám pulzus mindkét korlátot sérti, a 60 átmegy). A kimondott érvényességi feltétel (`is_valid`) Pythonban és C-ben generálódik; az ipari vezérlőre nem hordozható, ott külön logikai jel kell. A kerekítésnél a $0{,}1 + 0{,}2 \leq 0{,}3$ egzakt számolásban nem korlátoz (figyelmeztetés), 64 bites lebegőpontos számolásban soha nem teljesül (hibát jelez): a bizonyításnak ugyanazt a számábrázolást kell használnia, mint a gépnek.

4. Az igazolási dosszié. Rögzíti a specifikáció ujjlenyomatát (a szöveg számított azonosítóját), az időt és az eszköz verzióját, veszélyenként a kizáró szabályt, és saját hatókör-szakaszát. Nem tanúsítvány és nem független ellenőrzés (P4). A példában a jelek típusa és tartománya ismeretlen (a lapos nyelvtan nem ad meg tartományt), ami gyengébb állítás, a biztonságos állapot pedig nem ellenőrizhető.

Ismétlés típusos specifikációval (2026-11-01, Studio 2.12). Ugyanezt a három futtatást megismételtük a jeleket iránnyal, típussal, egységgel és tartománnyal leíró (típusos) fájlformátumban: az eredmény ugyanaz (megfelel, hiba a megnevezett veszélynél, megfelel a javítás után), a dosszié viszont már kitölti a jelek táblázatát, és a biztonságos állapotot is ellenőrzi. Mérési megfigyelés: a meg nem adott kimenet a biztonságos állapotban 0-ra áll, ami az intervallum-szabályt (legalább 250 ms) sértette, miközben a biztonságos állapot ellenőrzése megfelelt; ezért a biztonságos állapotban minden szabályozott kimenetet ki kell mondani. A fenti 1–4. pont megállapításai a régi, jelek nélküli formátumra vonatkoznak.

5. Meglévő programból váz. Az eszköz egy meglévő program szerkezetét (szenzor, aktuátor, talált számok) kiolvassa, de a limiteket és a veszélyt az embernek kell megadnia, nem a kódból örökölni. Saját fájljainkon az import csak részben ismerte fel a jeleket, mert névmintákra épít.

Mit nem tud (még). Nincs teljesség (P1: a veszélyeket és a szabályokat az ember mondja ki), nincs megkerülhetetlenség (P2a: a kapu csak a kijelölt határon kötelez; az operációs rendszer által kikényszerített fok (a kikényszerítés erőssége, nem a rendszerek szintje) Linuxon külön csomag, a hardverírás üres váz), és nincs független ellenőrzés (P4). A részletek a kiegészítő előadásban (`kiegeszito.html`) és az eszköz saját hatókör-dokumentumában vannak.

Szintek közötti kapcsolat

A sejtek helyi szabályaiból nem következik a közösség szabálya. A következtetéshez a közösségnek saját alkotmány kell, a sejteknek pedig ahhoz illő szerződés.

1. Helyi nem azonos a közössel. Két sejt közös készletből él, a készlet 8. Mindkettő betartja a saját szabályát (legfeljebb 5), mégis a Z3 ellenpéldát talált: $4 + 5 = 9 > 8$. A közösségi tulajdonság (emergencia) a sejtek szabályaiból nem vezethető le.

2. Garancia–feltevés. A közösség kvótát oszt (a kvóták összege legfeljebb a készlet), mindegyik sejt a saját kvótája alatt marad. Ebből következik, hogy a használat összege legfeljebb a készlet. Bizonyított 2, 3, 5 és 8 sejtre; tetszőleges számra az egyenlőtlenségek összeadásával.

3. A lánc ereje. Ha egyetlen sejt kapuja hiányzik, a közösségi garancia megdől (ellenpélda). A P2a előfeltevés tehát **minden szinten** külön teljesülnie kell.

4. Közösségi életképesség. Ha három sejtnek legalább 3-3-3 egység kell az élethez: 9 egységes készletből lehetséges az elosztás, 8-ból **bizonyítottan minden elosztás kizárt**. Ekkor a matematika nem dönt: az alkotmánynak először kell kimondania, kinek van előnye. Ez értékválasztás (P1), nem számítási kérdés.

5. Sebesség és koncentráció. Az ellátás egy lépésben 2-vel csökken (9-ről 7-re). Minden sejt lépésenként legfeljebb 1-gyel tud visszavenni, összesen hármal, ami több, mint 2. Mégis van ellenpélda: ha egy sejt 6,5-et használ (a többi 1 és 1,5), az összes visszavétel után is 7,5 marad. A **koncentrációs korlát** (egy sejt legfeljebb 4-et használhat) mellett a Z3 bizonyítja a megoldhatóságot. A korábbi „környezeti sebesség" feltétel tehát közösségi szinten egy második feltétellel egészül ki: **az alkalmazkodási képesség nem lehet egyetlen szereplőnél összpontosítva**. Ez a trösztellenes szabály formális megfelelője.

Vizsgálat	Állítás	Eredmény
1	helyi szabályból következik a közös	ellenpélda
2	kvóta + betartás ⇒ közös	bizonyított
3	hiányzó kapu mellett is áll a közös	ellenpélda
4	3-3-3 minimumhoz 8 elég	lehetetlen (bizonyított)
5a	gyors csökkenés korlát nélkül	ellenpélda
5b	gyors csökkenés koncentrációs korláttal	bizonyított

Metanyelv: miért használható a `.bio`-ra, és hogyan értendő

A `.bio` szabályai tárgynyelven szólnak a határról; a metanyelv az, amelyen e szabályokról állítunk. A különbség ugyanaz, mint a beszéd és a nyelvtan között, amelyből az egész gondolatmenet kiindult.

Mi a különbség? Tárgynyelven mondjuk: „a pulzus legfeljebb 165". Metanyelven mondjuk: „ez a szabálygyűjtemény ellentmondásmentes", vagy „ebből a két szabályból következik a közösségi szabály". Tarski kimutatta, hogy egy kellően gazdag nyelv igazságfogalmát csak egy másik, gazdagabb nyelvben lehet pontosan meghatározni. A nyelv nyelvtanának leírása is metanyelv: a `.bio` esetében a nyelvtanleíró fájl a `.bio`-ról szól, nem `.bio`-ban íródott. A fájl jelenlegi (1.0) változata szűkebb nyelvet ír le, mint amelyet a `pacemaker.bio` használ.

Miért használható a `.bio`-ra? Négy ok:

1. A `.bio` szándékosan korlátozott (véges, eldönthető). Ezért a róla szóló állítások (van-e zsákutca, következik-e a közös szabály) a gép számára teljesen eldönthetők.
2. A szintek kapcsolata maga is metanyelvi. A közösségi szabály a sejtek szabályainak *paramétereiről* szól (a kvóták összege), tehát alkotmány az alkotmányokról.
3. A rétegezés elkerüli az önhivatkozás paradoxonát (Russell, Tarski): minden szint csak a nála alacsonyabb szintről állít.
4. Kettéválasztja a szabály és a bizonyíték szerepét. A sejt a tárgynyelven él, a bizonyítás a metanyelven történik, ahogy az anyanyelvi beszélő a nyelvtantól függetlenül beszél.

Hogyan értendő? A metanyelvi állítás nem parancs a sejtnek, nem magasabb rendű hatalom, és nem része a tárgynyelvnek. Állítás a szabályokról, amelynek igazsága ellenőrizhető. A sejthez nem ad hozzá új képességet.

A Gödel–Tarski-korlátok azokra a nyelvekre vonatkoznak, amelyek a saját mondataikat számokká tudják kódolni. A `.bio` ezt nem tudja, a róla szóló állításokhoz pedig a valós számok elmélete elegendő, amely eldönthető (Tarski). Ezért itt nem kell végtelen metanyelv-létrát felmászni.

Az ellenkező:

Ellenérv	Minősítés
Végtelen regresszus: ki ellenőrzi az ellenőrzőt? A Z3 is hibázhat.	**Áll részben.** Az ellenőrzés tanúsítvánnyal egy kis, független programmal ellenőrizhető, a hibalehetőség egy szűk bizalmi magra szűkíthető, de nem szüntethető meg. Ez egy kimondott előfeltevés.
Gödel–Tarski: egy elég erős, önmagát kikódolni tudó rendszer nem bizonyíthatja a saját ellentmondásmentességét, és a saját igazságát sem definiálhatja.	**Nem áll** erre a nyelvre (nincs benne önkódolás). Ha bővítenénk, a rétegezés kötelező lenne.
Pontatlan metanyelv: a magyar szöveg többértelmű lehet.	**Áll.** Ezért van a szószedet, és a kritikus állítások gépi (Z3) alakban is megvannak.
A metanyelv előbb kell, mint a tárgynyelv.	**Nem áll.** A nyelv a nyelvtan nélkül is létezik; a sorrend történeti, nem logikai.

Ellenbizonyítás: a tagadás alakjai és a mérleg

A tézis szigorú tagadása megdőlt, de több gyengébb tagadás megmaradt, és ezek adják a hatókört. Minden vizsgálat ugyanazt a módszert követi: az állítás tagadását kérdezzük a géptől, és vagy konkrét ellenpéldát kapunk, vagy bizonyítást, hogy nincs.

#	Tagadás	Módszer	Eredmény	Következmény
1	Van doboz és kapu, amelynél a kimenet kilép E-ből (valós számokon)	Z3	nincs ellenpélda	a tétel a modellben igaz
2	Ugyanez gépi lebegőpontos számokon	Z3, 2^{64} bitminta	naiv kapu: ellenpélda („nem szám”); javított: bizonyított	a megvalósítási rés egzakt ellenőrzéssel zárható
3	A specifikáció a saját vészállapotával ellentmond	Z3	igen: az erős kiút-feltétel hamis	az életképességet vizsgálni kell
4	A kapu csak biztonságot ad, optimumot nem	a nyelvtan tartományt ismer	nem áll, ha az optimum ismert tartomány	marad a környezeti sebességfeltétel (0,02 / mp bizonyított, 0,03 ellenpélda)
5	A helyi szabályból következik a közös	Z3	ellenpélda	a szintnek saját alkotmány kell
6	Hiányzó kapu mellett is áll a közös garancia	Z3	ellenpélda	P2a minden szinten
7	A közösségi életképesség mindig biztosítható	Z3	8 készletből 3-3-3 minimummal lehetetlen	értékválasztás (P1)
8	Gyors alkalmazkodás koncentráció nélkül is megoldható	Z3	ellenpélda; korláttal bizonyított	koncentrációs korlát
9	A metanyelv végtelen regresszus / Gödel	érvelés	részben áll / nem áll	véges bizalmi mag, kimondva
10	A valóságra vonatkozó 100%	Hume	nem bizonyítható	P3
11	A specifikáció helyessége	–	nem bizonyítható	P1
12	Nyílt végű változás: új változók jelennek meg (Kauffman)	érvelés	vitatott, nem tétel	a változólista egy adott határig érvényes
13	Vibe-codinghoz ismerni kell a programnyelvet	érvelés a kapu-tételből	nem áll a hatás garantálására; a szakterület ismerete kell	a szabály helyessége (P1) módszerfüggetlen
14	A soronkénti kódolás vagy olvasás biztonságosabb	érvelés	nem áll: ott a teljes kód a bizalmi mag, a kapunál csak a kapu és az ellenőrző	a bizalmi mag méretéről szól, nem a hibák valószínűségéről
15	A gép magától megtalálja a hiányzó szabályt	Z3, a Studio élő futtatása	nem áll: veszély megnevezése nélkül megfelel; megnevezett veszélynél ellenpélda	a veszélyek felsorolása szakterületi tudás (P1)
16	Egy specifikációból minden nyelven azonos a viselkedés	generálás három nyelvre	nem áll: a szabályok azonosak, a viselkedés nem (az ipari vezérlő és a C csak jelez, mert nincs kimeneti jel)	a kimeneti jelet ki kell mondani (P2b)
17	A kerekítés nem számít	Z3, egzakt és 64 bites lebegőpontos mód	nem áll: a $0{,}1 + 0{,}2 \leq 0{,}3$ egzaktul nem korlátoz, lebegőpontosan soha nem teljesül	a bizonyításnak a gép számábrázolását kell követnie (P4)

Bizonyított (a megadott előfeltevések mellett): az 1., 2. (javított kapura), 4. (a sebességfeltétellel), 5–8. sor eredményei. **Nem bizonyítható, mert nem ténykérdés vagy megfigyelés**: a 10. és 11. sor.

Nyitott: a 12. sor. A 13. és 14. sor érvelés: a bizalmi mag méretéről szól, nem valószínűségről. A 15–17. sor a MetaSpace Studion mért eredmény (2026-10-30), lásd az „Egy eszközön végigvíve” szakaszt.

Előfeltevések, hatókör, nyitott kérdések

Előfeltevések. A tételek ezekből és csak ezekből következnek:

- **P1:** a cél (E) helyes. Értékválasztás; a biológiában adott (fennmaradás).
- **P2a:** nincs mellékút, minden szinten.
- **P2b:** a határ teljes (látható és befolyásolható rajta a lényeges).
- **P3:** a fizika és a környezet együttműködik, környezeti sebességgel és koncentrációval együtt.
- **P4:** az ellenőrző (Z3) és a fordító megbízható, vagyis a bizalmi mag helyes.

Hatókör. A `.bio` nem a világ törvénykönyve, hanem egy határolt rendszer határának alkotmánya: kijelölt lényeges változókra, szintenként külön. A világ egészének nincs külseje, ezért nincs hová a kaput állítani, amelyet a belső nem tud megkerülni: az alkotmány csak a világ egy részére mondható ki.

Nyitott kérdések:

1. A P2b ellenőrzése a pacemakeren: mely belső változók nincsenek kivezetve.
2. A sebességfeltétel általános levezetése tetszőleges sávmozgásra.
3. A közösségi szint többlépéses, dinamikus modellje.
4. A bizalmi mag minimalizálása (tanúsítványt ellenőrző kis program).
5. Nyílt végű változás: új változók felvétele a szerződésbe futás közben.

Mellékeredmény a MetaSpace-nek. A `pacemaker.bio`-ból hiányzik a vészállapoti határidő és a lefelé irányú változás korlátja; a `pacemaker_guardrail.py` olvasat szerint nem korlátozza az alsó határt, és a NaN átmegy rajta (kódolvasás, nem futtatva). Javítandó a befektetői demó előtt. Állapot (2026-10-28 és 10-30): javított másolatok készültek (pacemaker_javitott.bio, pacemaker_guardrail_javitott.py: alsó korlát, pozitív igazolás, őrkutya), az eredetik érintetlenek; a fordító és a telepített zár a határidőt továbbra sem kényszeríti ki, a futási kernel gyorsulás-korlátja pedig 100 ezredmásodperces lépésre számol, nem másodpercre.

Függelék: gépi bizonyítások és hivatkozások

A gépi vizsgálatok a `_hinta_eloadas` mappában futtathatók (Python, z3-solver csomag).

Fájl	Tartalma
`pacemaker_kiut_feltetel.py`	életképesség lépcsői, kapu-tétel, lebegőpontos kapu, optimális sáv
`szintek_kapcsolata.py`	helyi és közös, kvóta-szerződés, hiányzó kapu, közösségi életképesség, koncentráció
`ALLAPOT.md`	a munka állása, előfeltevések, helyesbítések
diak.html	a 21 diás előadás (internet nélkül nyitható)
kiegeszito.html	a 8 diás kiegészítő előadás a MetaSpace Studióról (élő demóval, mért eredményekkel)
dosszie_pacemaker.html	a Studio igazolási dossziéja a javított pacemaker-specifikációról (2026-10-30)

Minden vizsgálat formája: `unsat` a tagadásra = bizonyított; `sat` = konkrét ellenpélda.

Hivatkozások (a kiadási adatok ellenőrizve: 2026-11-01, kiadói és könyvtári katalógusok alapján):

- Ryle, G.: *The Concept of Mind* (1949); Polanyi, M.: *The Tacit Dimension* (1966); Wittgenstein, L.: *Filozófiai vizsgálódások*.

- Bernard, C.: a belső környezet állandóságáról (előadásai 1855–56-tól; összefoglalása: Leçons sur les phénomènes de la vie, 1878); Ashby, W. R.: *Design for a Brain* (1952); Conant, R. és Ashby, W. R. (1970): a jó szabályozó tétele.
- Maturana, H. és Varela, F.: *Autopoiesis and Cognition* (1980); Aubin, J.-P.: *Viability Theory* (1991).
- Tarski, A.: az igazság fogalmáról formális nyelvekben (1933, lengyelül; 1935, németül); *A Decision Method for Elementary Algebra and Geometry* (RAND, 1948; 2. kiadás: University of California Press, 1951).
- Rice, H. G. (1953); Angluin, D.: *Learning Regular Sets from Queries and Counterexamples* (1987); Kauffman, S.: *Investigations* (2000); Hume, D.: *Treatise of Human Nature* (1739–40).

Bevett nevek forrásai (2026-11-06, webes kereséssel ellenőrizve: szerző, év, folyóirat). A szószedetben szereplő Reiter (1978), Putnam és Fodor, Ryle (1949), Polanyi (1966), Aubin adatai ebben a körben nem lettek újraellenőrizve.

- Schneider, F. B. (2000). Enforceable security policies. *ACM Transactions on Information and System Security* 3(1), 30–50. (A futásidő figyeléssel kikényszeríthető szabályok a biztonsági tulajdonságok részhalmaza.)
- Alpern, B., Schneider, F. B. (1985). Defining liveness. *Information Processing Letters* 21(4), 181–185.
- Lamport, L. (1977). Proving the correctness of multiprocess programs. *IEEE Transactions on Software Engineering* SE-3(2), 125–143. (A biztonság és az élősség megkülönböztetése; a formális definíciók későbbiek.)
- Sha, L. (2001). Using simplicity to control complexity. *IEEE Software* 18(4), 20–28. (Simplex-architektúra.)
- Alshiekh, M., Bloem, R., Ehlers, R., Könighofer, B., Niekum, S., Topcu, U. (2018). Safe reinforcement learning via shielding. *AAAI-18*, 2669–2678.
- Saltzer, J. H., Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE.* (Alapértelmezett tiltás, teljes közvetítés.)
- Anderson, J. P. (1972). Computer security technology planning study (ESD-TR-73-51). (A referenciafigyelő fogalma; a kiadás éve forrásonként 1972 vagy 1973.)
- Karpathy, A. (2025. február 2.). Bejegyzés az X-en: a „vibe coding” kifejezés eredete.